FASE 2 · APLICAÇÃO DE PADRÕES GOF

Aplicação de Padrões de Projeto GoF

Documentação da implementação dos padrões Singleton, Factory e Strategy no protótipo PizzaGest

Disciplina: IES300 – Engenharia de Software III · Prof.ª Elisabete Ferreira dos Santos Gomes Silva
Projeto: PizzaGest – Sistema de Gestão de Pedidos para Delivery Local · **Fase 2:** Implementação e Padrões de Projeto
Repositório: github.com/B3rnardi/restaurante-projeto-eng-III (commit `3357f90`, arquivo `pizzagest.py`)
Integrantes: Luiz Miguel Teles, Cauã Vieira Alves, Leonardo Soares Alcebíades, Fernando de Bernardi Camargo

1. Visão geral

O protótipo PizzaGest implementa o núcleo do sistema de pedidos definido na Fase 1: cadastro de produtos do cardápio, registro de pedidos com seus itens, cálculo de valores, fechamento do pedido e emissão do relatório de vendas restrito ao administrador. O código está organizado nas três camadas previstas na definição arquitetural (Dados, Negócio e Apresentação), e cada um dos três padrões GoF foi aplicado na camada em que resolve um problema concreto do domínio.

Padrão	Categoria GoF	Classes envolvidas	Camada	Problema que resolve
Singleton	Criacional	`BaseDadosPizzariaSingleton`	Dados	Ter uma única base de dados e um único gerador de número de pedido para todo o sistema.
Factory	Criacional	`ProdutoFactory`, `Produto`, `Pizza`, `Bebida`	Negócio	Criar o tipo certo de produto a partir de um código (“PIZZA”, “BEBIDA”) sem espalhar `if`s pelo sistema.
Strategy	Comportamental	`CalculoPrecoStrategy`, `PrecoMaiorValorStrategy`, `PrecoMediaStrategy`, `Pedido`	Negócio	Permitir trocar a regra de preço da pizza meio a meio sem alterar a classe `Pedido`.

As referências de linha deste documento correspondem ao arquivo `pizzagest.py` no commit indicado na capa.

2. Singleton Criacional

> **Intenção (GoF):** garantir que uma classe tenha somente uma instância e fornecer um ponto global de acesso a ela.

2.1 Problema no sistema

Vários objetos precisam do mesmo repositório de dados: o `Pedido` precisa de um número sequencial e de um lugar para ser gravado quando é fechado, e o `ServicoPizzariaController` precisa ler os produtos, os usuários e os pedidos para fazer login, cadastrar produtos e gerar o relatório. Se cada um criasse sua própria base, o relatório não enxergaria os pedidos gravados e dois pedidos poderiam receber o mesmo número.

2.2 Estrutura

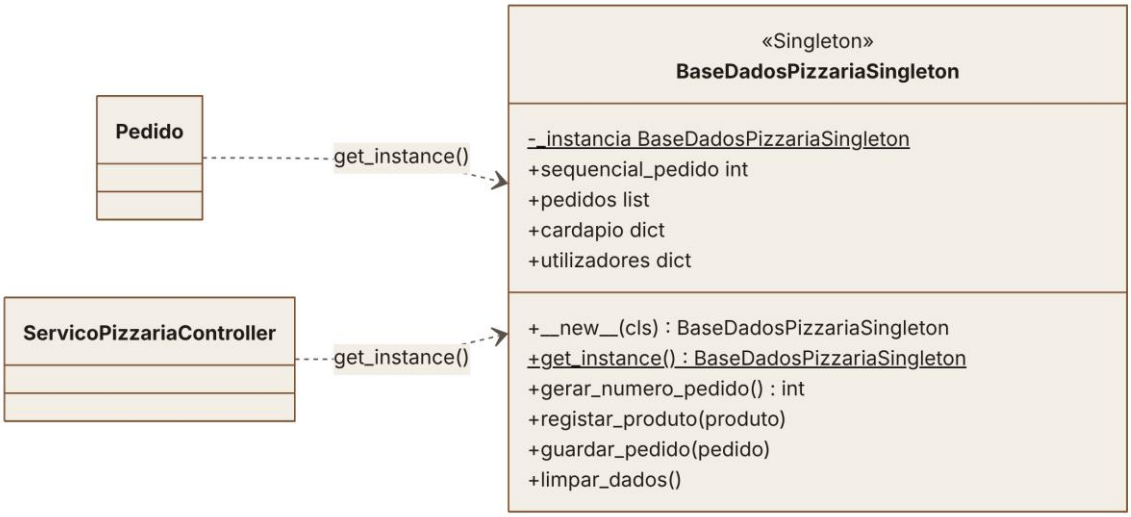


Figura 1 – Diagrama de classes do Singleton e de seus clientes.

Participante GoF	Implementação no PizzaGest
Singleton	`BaseDadosPizzariaSingleton` (linha 8)
Instância única (atributo de classe)	`_instancia = None` (linha 13)
Controle da criação	`__new__` só cria o objeto na primeira chamada (linhas 15–19)
Ponto global de acesso	`get_instance()` (linhas 30–32)
Clientes	`Pedido.__init__` (linha 140), `Pedido.fechar_pedido` (linha 167), `ServicoPizzariaController.__init__` (linha 173)

2.3 Implementação

pizzagest.py – BaseDadosPizzariaSingleton

```
 8  class BaseDadosPizzariaSingleton:
 9      """
10      Padrão GoF SINGLETON: Garante uma única instância para um recurso
11      partilhado (base de dados em memória e gerador de senhas de pedidos).
12      """
13      _instancia = None
14
15      def __new__(cls):
16          if cls._instancia is None:
17              cls._instancia = super().__new__(cls)
18              cls._instancia._inicializar()
19          return cls._instancia
    ⋮
30      @classmethod
31      def get_instance(cls):
32          return cls()
33
34      def gerar_numero_pedido(self) -> int:
35          self.sequencial_pedido += 1
36          return self.sequencial_pedido
```

Em Python, o construtor `__new__` é chamado antes do `__init__` e é quem efetivamente aloca o objeto. Sobrescrevê-lo permite devolver sempre a mesma instância, mesmo que alguém escreva `BaseDadosPizzariaSingleton()` diretamente. A inicialização dos dados fica em `_inicializar()` (e não em `__init__`) para que ela rode uma única vez: se estivesse no `__init__`, cada nova chamada apagaria os pedidos já gravados.

pizzagest.py – pontos de uso

```
140         self.numero = BaseDadosPizzariaSingleton.get_instance().gerar_numero_pedido()
    ⋮
167         BaseDadosPizzariaSingleton.get_instance().guardar_pedido(self)
    ⋮
173         self.db = BaseDadosPizzariaSingleton.get_instance()
```

2.4 Consequências

- **Benefícios:** número de pedido sequencial e sem repetição; todos os módulos enxergam os mesmos dados; ponto único para trocar a base em memória por um banco real nas próximas fases.
- **Cuidados:** estado global dificulta testes isolados, o que foi resolvido com `limpar_dados()` (linha 44), chamado no `setUp` de cada teste. A implementação não é *thread-safe*; se o sistema virar uma API com requisições simultâneas, a criação da instância e o `gerar_numero_pedido()` devem ser protegidos com `threading.Lock`.

Sem o padrão: seria preciso criar a base no início do programa e passá-la manualmente para cada classe, ou cada classe criaria a sua — e os pedidos gravados por uma não apareceriam no relatório da outra.

3. Factory Criacional

> **Intenção (Factory Method, GoF):** definir uma interface para criar um objeto, deixando que outra parte do código decida qual classe concreta instanciar. O PizzaGest usa a forma simplificada conhecida como **Simple Factory**: uma classe com um método estático de criação que centraliza a decisão sobre qual subclasse de `Produto` devolver.

3.1 Problema no sistema

O cardápio tem categorias diferentes de produto (pizzas e bebidas), cada uma representada por uma subclasse de `Produto`. Quem cadastra um produto recebe apenas um texto com o tipo, vindo da entrada do usuário. Sem um ponto central de criação, o controlador — e qualquer outro módulo que crie produtos — precisaria conhecer todas as subclasses e repetir o mesmo encadeamento de `if/elif`.

3.2 Estrutura

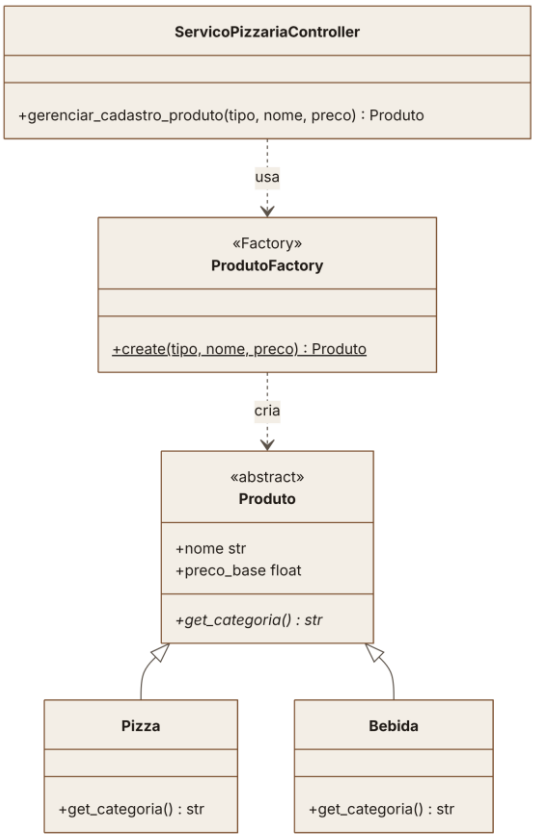


Figura 2 – Diagrama de classes da Factory.

Participante	No PizzaGest
Produto (abstrato)	`Produto(ABC)` com o método abstrato `get_categoria()` (linhas 51–62)
Produtos concretos	`Pizza` (linha 65) e `Bebida` (linha 70)
Fábrica	`ProdutoFactory.create()` (linhas 75–90)
Cliente	`ServicoPizzariaController.gerenciar_cadastro_produto()` (linhas 181–184)

3.3 Implementação

pizzagest.py – ProdutoFactory

```
75  class ProdutoFactory:
76      """
77      Padrão GoF FACTORY: Cria objetos (Pizza ou Bebida) sem expor
78      a lógica de instanciação ao cliente.
79      """
80      @staticmethod
81      def create(tipo: str, nome: str, preco: float) -> Produto:
82          if not tipo:
83              raise ValueError("O tipo de produto não pode ser vazio.")
84          tipo_normalizado = tipo.strip().upper()
85          if tipo_normalizado == "PIZZA":
86              return Pizza(nome, preco)
87          elif tipo_normalizado == "BEBIDA":
88              return Bebida(nome, preco)
89          else:
90              raise ValueError(f"Tipo de produto desconhecido: {tipo}")
```

A fábrica normaliza o tipo informado (`strip().upper()`), aceita “pizza”, “ Pizza ” ou “PIZZA” e rejeita tipos vazios ou desconhecidos com `ValueError`. As validações comuns (nome obrigatório, preço maior que zero) ficam no construtor de `Produto` (linhas 52–58), de modo que qualquer produto criado pela fábrica já nasce válido.

pizzagest.py – uso da fábrica pelo controlador

```
181     def gerenciar_cadastro_produto(self, tipo: str, nome: str, preco: float) -> Produto:
182         produto = ProdutoFactory.create(tipo, nome, preco)
183         self.db.registar_produto(produto)
184         return produto
```

3.4 Consequências

- **Benefícios:** o controlador depende apenas da abstração `Produto`; a regra "qual classe criar" fica em um só lugar; incluir uma nova categoria (por exemplo, `Sobremesa`) exige criar a subclasse e uma linha na fábrica, sem mexer no controlador, no pedido ou no relatório.
- **Cuidados:** por ser uma Simple Factory, a fábrica ainda precisa ser editada a cada nova categoria. Se o número de categorias crescer, ela pode evoluir para um dicionário de registro (`{"PIZZA": Pizza, "BEBIDA": Bebida}`) ou para o Factory Method completo, com uma fábrica por categoria.

4. Strategy Comportamental

> **Intenção (GoF):** definir uma família de algoritmos, encapsular cada um deles e torná-los intercambiáveis. O Strategy permite que o algoritmo varie independentemente dos clientes que o utilizam.

4.1 Problema no sistema

Na pizza meio a meio, o preço depende de uma regra comercial: normalmente cobra-se o sabor mais caro, mas em promoções pode-se cobrar a média dos dois sabores. Essa regra muda com o tempo (novas promoções, dias especiais) e não deveria obrigar a alterar a classe `Pedido`, que já cuida de itens, totais e fechamento.

4.2 Estrutura

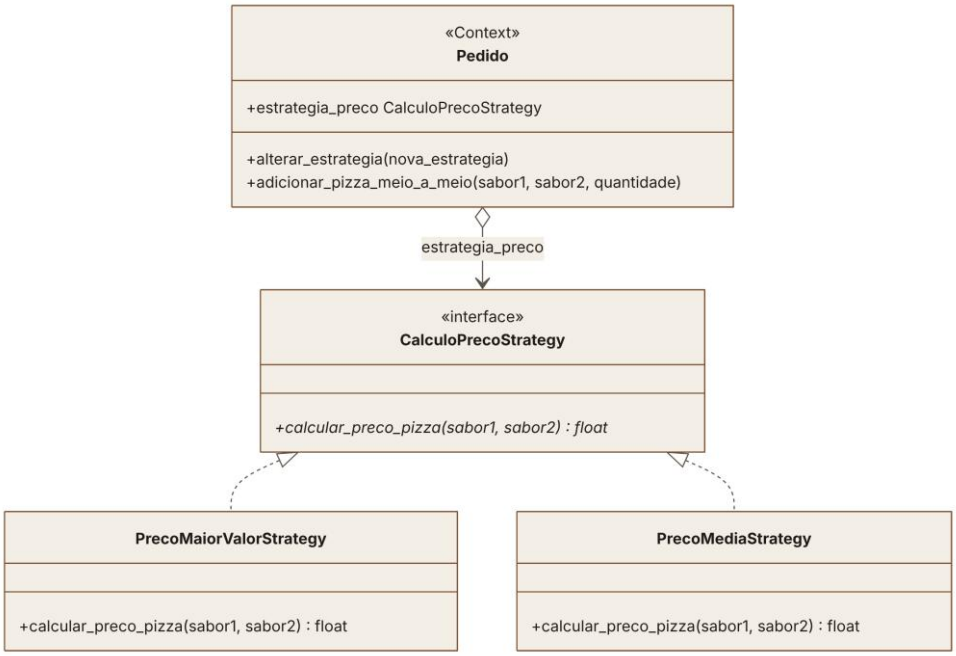


Figura 3 – Diagrama de classes do Strategy.

Participante GoF	Implementação no PizzaGest
Strategy (interface)	`CalculoPrecoStrategy`, classe abstrata com `calcular_preco_pizza()` (linhas 96–103)
ConcreteStrategy	`PrecoMaiorValorStrategy` – regra padrão (linhas 106–109) `PrecoMediaStrategy` – regra promocional (linhas 112–115)

Participante GoF	Implementação no PizzaGest
Context	`Pedido`, que guarda a estratégia em `estrategia_preco` (linha 145) e a usa em `adicionar_pizza_meio_a_meio()` (linha 155)

4.3 Implementação

pizzagest.py – família de estratégias

```
 96  class CalculoPrecoStrategy(ABC):
 97      """
 98      Padrão GoF STRATEGY: Define uma família de algoritmos de precificação
 99      permitindo trocá-los sem alterar o contexto do pedido.
100      """
101      @abstractmethod
102      def calcular_preco_pizza(self, sabor1: Produto, sabor2: Produto) -> float:
103          pass
104
105
106  class PrecoMaiorValorStrategy(CalculoPrecoStrategy):
107      """Cobra pelo sabor mais caro na pizza meio a meio (regra padrão)."""
108      def calcular_preco_pizza(self, sabor1: Produto, sabor2: Produto) -> float:
109          return max(sabor1.preco_base, sabor2.preco_base)
110
111
112  class PrecoMediaStrategy(CalculoPrecoStrategy):
113      """Cobra a média simples entre os dois sabores (regra promocional)."""
114      def calcular_preco_pizza(self, sabor1: Produto, sabor2: Produto) -> float:
115          return (sabor1.preco_base + sabor2.preco_base) / 2.0
```

pizzagest.py – Pedido como contexto

```
137      def __init__(self, cliente: str, estrategia_preco: CalculoPrecoStrategy = None):
   ⋮
145          self.estrategia_preco = estrategia_preco or PrecoMaiorValorStrategy()
146
147      def alterar_estrategia(self, nova_estrategia: CalculoPrecoStrategy):
148          self.estrategia_preco = nova_estrategia
   ⋮
154      def adicionar_pizza_meio_a_meio(self, sabor1: Produto, sabor2: Produto, quantidade: int):
155          preco = self.estrategia_preco.calcular_preco_pizza(sabor1, sabor2)
156          descricao = f"Pizza 1/2 {sabor1.nome} + 1/2 {sabor2.nome}"
157          item = ItemPedido(descricao, quantidade, preco)
158          self.itens.append(item)
```

O `Pedido` recebe a estratégia no construtor; se nenhuma for informada, usa a regra padrão `PrecoMaiorValorStrategy` (linha 145). A estratégia pode ainda ser trocada durante a montagem do pedido com `alterar_estrategia()`. Em nenhum momento o `Pedido` sabe qual regra está sendo aplicada: ele só chama `calcular_preco_pizza()`.

Como seria sem o padrão (não utilizado no projeto)

```
def adicionar_pizza_meio_a_meio(self, sabor1, sabor2, quantidade, regra):
    if regra == "MAIOR_VALOR":
        preco = max(sabor1.preco_base, sabor2.preco_base)
    elif regra == "MEDIA":
        preco = (sabor1.preco_base + sabor2.preco_base) / 2
    # cada nova promoção = mais um elif dentro de Pedido
```

4.4 Consequências

- **Benefícios:** cada regra de preço é uma classe pequena, testável isoladamente; novas regras (ex.: preço proporcional, desconto de terça-feira) entram como novas classes, sem modificar `Pedido` — princípio Aberto/Fechado.
- **Cuidados:** quem cria o pedido precisa escolher a estratégia adequada; hoje essa escolha está no protótipo de console, e nas próximas fases pode ser movida para o controlador (por exemplo, com base em uma promoção ativa).

5. Colaboração entre os padrões

Os três padrões atuam juntos no fluxo principal do sistema. A Figura 4 mostra o caminho de um pedido do protótipo: o produto é criado pela **Factory** e gravado no **Singleton**; o pedido obtém seu número no **Singleton**, calcula o preço da pizza meio a meio pela **Strategy** escolhida e, ao ser fechado, é gravado no **Singleton**, de onde o relatório o lê.

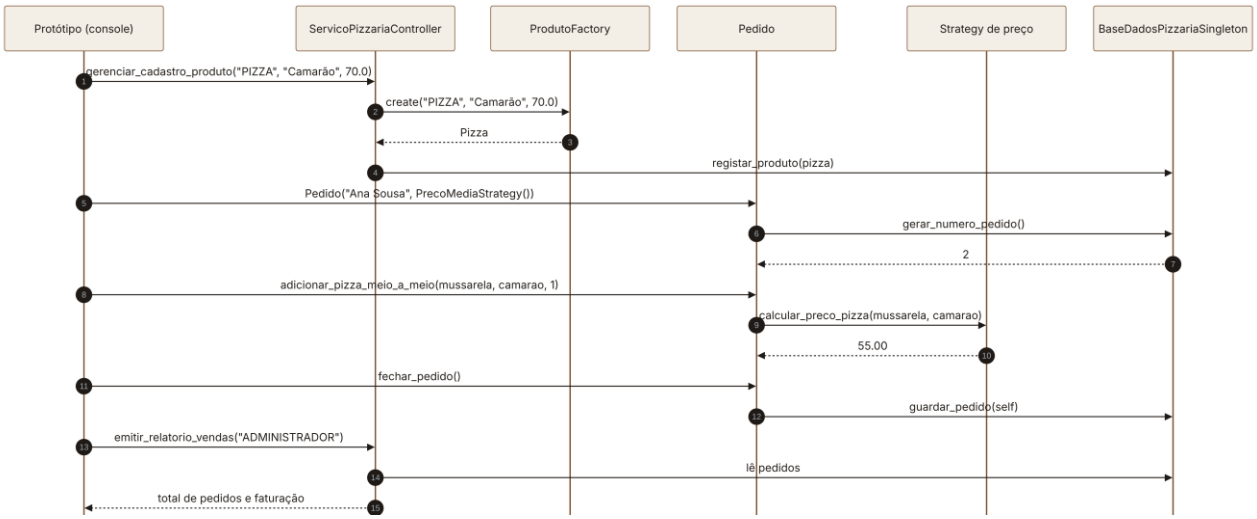


Figura 4 – Diagrama de sequência do cadastro de produto, registro de pedido e emissão de relatório.

6. Evidência de funcionamento

Os testes automatizados de `test_pizzagest.py` exercitam os três padrões: CT03 compara as duas estratégias de preço (R$ 70,00 pelo maior valor e R$ 55,00 pela média, para Mussarela R$ 40,00 + Camarão R$ 70,00) e CT04 integra Factory, Strategy, Pedido, Singleton e relatório. Resultado da execução:

python -m unittest -v test_pizzagest

```
test_01_entrada_valida (test_pizzagest.TestPizzaGest.test_01_entrada_valida)
CT01 - Entrada válida: Calcula pedido com pizza inteira e bebidas. ... ok
test_02_entrada_invalida (test_pizzagest.TestPizzaGest.test_02_entrada_invalida)
CT02 - Entrada inválida: Rejeita quantidade negativa ou zero. ... ok
test_03_regra_de_negocio_strategy (test_pizzagest.TestPizzaGest.test_03_regra_de_negocio_strategy)
CT03 - Regra de negócio: Pizza meio a meio pelo maior valor e pela média. ... ok
test_04_integracao_entre_modulos (test_pizzagest.TestPizzaGest.test_04_integracao_entre_modulos)
CT04 - Integração: Factory + Strategy + Pedido + Singleton + Relatório. ... ok
test_05_tratamento_de_erro_pedido_vazio (test_pizzagest.TestPizzaGest.test_05_tratamento_de_erro_pedido_vazio)
CT05 - Tratamento de erro: Impede o fecho de um pedido sem itens. ... ok

----------------------------------------------------------------------
Ran 5 tests in 0.000s

OK
```

python pizzagest.py

```
============================================================
 BEM-VINDO(A) AO SISTEMA PIZZAGEST (PROTÓTIPO)
============================================================
[LOGIN] Autenticado com sucesso! Perfil: ADMINISTRADOR

RESUMO GERAL E ATIVIDADES RECENTES:
Total de Pedidos Fechados: 2
Faturação Total: R$ 149.00

Pedido #1 | Cliente: Carlos Mendes | Status: FECHADO
  - 1x Pizza 1/2 Mussarela + 1/2 Camarão: R$ 70.00
  - 2x Refrigerante 2L: R$ 24.00
  Total do Pedido: R$ 94.00

Pedido #2 | Cliente: Ana Sousa | Status: FECHADO
  - 1x Pizza 1/2 Mussarela + 1/2 Camarão: R$ 55.00
  Total do Pedido: R$ 55.00
```

Os pedidos #1 e #2 têm a mesma pizza (Mussarela + Camarão), mas valores diferentes (R$ 70,00 e R$ 55,00), porque cada um foi criado com uma estratégia de preço diferente. A numeração sequencial (#1, #2) e o relatório com os dois pedidos confirmam que todos os módulos compartilham a mesma instância da base de dados.